



UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

ALINE DE BRITO SÉRIO

MÉTODO DA JANELA DINÂMICA NO COPPELIASIM

JUAZEIRO - BA

2026

**UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO**

ALINE DE BRITO SÉRIO

MÉTODO DA JANELA DINÂMICA NO COPPELIASIM

Projeto apresentado à Universidade Federal do Vale do São Francisco– UNIVASF, Campus Tecnológico, como requisito para obtenção de nota na disciplina de ROBÓTICA I.

Orientador: Prof. JURACY EMANUEL MAGALHÃES DA FRANCA.

JUAZEIRO - BA

2026

1. Introdução

A navegação autônoma é uma das principais áreas estudadas dentro da robótica móvel. Um robô autônomo deve ser capaz de se locomover em um ambiente, identificar obstáculos e tomar decisões em tempo real para alcançar um objetivo sem causar colisões.

Dentro desse contexto, algoritmos de navegação local possuem grande importância, pois permitem que o robô realize desvios de obstáculos enquanto continua seguindo um alvo definido. Entre esses algoritmos, destaca-se o Dynamic Window Approach (DWA), bastante utilizado em aplicações de robótica móvel devido à sua capacidade de tomada de decisão em tempo real.

Neste projeto foi desenvolvido um sistema de navegação autônoma utilizando o método Dynamic Window Approach (DWA) integrado ao ambiente de simulação CoppeliaSim através da linguagem Python e da ZMQ Remote API.

O sistema desenvolvido permite que o robô:

- localize um alvo definido no cenário;
- navegue autonomamente;
- detecte obstáculos;
- realize desvios;
- saia de regiões presas;
- controle diferencialmente as rodas;
- interrompa a navegação ao alcançar o objetivo.

O projeto foi desenvolvido de forma incremental, passando por diversos ajustes práticos relacionados aos sensores, velocidades, controle angular e lógica de fuga.

2. Objetivos

2.1 Objetivo Geral

Implementar um sistema de navegação autônoma baseado no algoritmo Dynamic Window Approach (DWA) em um robô móvel simulado no CoppeliaSim.

2.2 Objetivos Específicos

- Integrar Python com o CoppeliaSim utilizando ZMQ Remote API;
- Controlar o movimento diferencial do robô;
- Utilizar sensores de proximidade;
- Detectar obstáculos em tempo real;
- Implementar perseguição ao Target;

- Desenvolver lógica de desvio de obstáculos;
- Criar um sistema de fuga para situações onde o robô fica preso;
- Encerrar automaticamente a simulação ao alcançar o objetivo.

3. Fundamentação Teórica

3.1 Robótica Móvel

A robótica móvel estuda sistemas robóticos capazes de se locomover autonomamente em ambientes diversos. Para isso, o robô precisa combinar sensores, algoritmos e controle de movimento para tomar decisões durante a navegação.

Entre os principais desafios da robótica móvel estão:

- localização;
- planejamento de trajetória;
- desvio de obstáculos;
- controle cinemático;
- tomada de decisão em tempo real.

3.2 Dynamic Window Approach (DWA)

O Dynamic Window Approach (DWA) é um algoritmo de navegação local desenvolvido por Dieter Fox, Wolfram Burgard e Sebastian Thrun.

O método busca selecionar velocidades lineares e angulares seguras considerando as limitações físicas do robô e os obstáculos presentes no ambiente.

O algoritmo avalia continuamente possíveis trajetórias futuras utilizando:

- velocidade linear;
- velocidade angular;
- direção até o objetivo;
- distância dos obstáculos;
- segurança da trajetória.

A principal vantagem do DWA é permitir navegação em tempo real mesmo em ambientes dinâmicos.

3.3 Sensores de Proximidade

Sensores de proximidade permitem detectar objetos próximos ao robô. Neste projeto, foram utilizados sensores posicionados em diferentes direções para ampliar o campo de percepção do ambiente.

Os sensores utilizados foram:

- sensor frontal;
- sensor diagonal esquerdo;
- sensor diagonal direito;
- sensor lateral esquerdo;
- sensor lateral direito.

Esses sensores foram fundamentais para:

- detectar obstáculos;
- evitar colisões;
- identificar regiões livres;
- auxiliar no sistema de fuga.

4. Tecnologias Utilizadas

As principais tecnologias utilizadas no desenvolvimento foram:

- Python 3;
- CoppeliaSim Edu;
- VS Code;
- ZMQ Remote API;

5. Desenvolvimento do Projeto

5.1 Construção do Cenário

O ambiente de simulação foi desenvolvido no CoppeliaSim.

O cenário continha:

- robô móvel;
- obstáculos;
- objeto Target;
- sensores de proximidade;
- plano de movimentação.

Os obstáculos foram posicionados estrategicamente para testar o comportamento do algoritmo em situações de desvio e aprisionamento.

5.2 Estrutura do Robô

O robô utilizado possui:

- duas rodas motrizes;
- movimentação diferencial;
- sensores de proximidade;
- controle independente das rodas.

O movimento do robô é realizado através da diferença de velocidade entre as rodas esquerda e direita.

5.3 Comunicação Python + CoppeliaSim

A comunicação entre Python e CoppeliaSim foi realizada utilizando a ZMQ Remote API.

Inicialmente foi necessário:

- estabelecer conexão;
- obter os motores;
- acessar os sensores;
- obter o objeto Target;
- controlar a simulação externamente.

A comunicação permitiu que o algoritmo fosse executado diretamente pelo VS Code enquanto o ambiente era simulado no CoppeliaSim.

5.4 Controle do Movimento

O sistema calcula continuamente:

- posição do robô;
- posição do Target;
- erro angular;
- distância até o objetivo.

Com base nesses dados são calculadas:

- velocidade linear (v);
- velocidade angular (w).

Esses valores são convertidos em velocidades das rodas.

Durante os testes foram realizados diversos ajustes nos limites de velocidade para melhorar:

- estabilidade;
- velocidade de navegação;
- precisão;
- controle das curvas.

5.5 Perseguição do Target

O robô foi programado para localizar continuamente o objeto Target dentro do cenário.

O sistema calcula:

- diferença de posição;
- direção do objetivo;
- erro angular.

A partir disso o robô ajusta sua rotação para permanecer apontado em direção ao alvo.

Após o alinhamento, o robô avança até atingir a região definida como objetivo.

5.6 Sistema de Desvio de Obstáculos

Durante a navegação os sensores verificam continuamente a presença de obstáculos.

Quando um obstáculo é detectado:

- o movimento normal é interrompido;
- o modo de fuga é ativado;
- o robô tenta encontrar uma direção livre.

O sistema analisa os sensores laterais e diagonais para decidir qual lado possui maior espaço disponível.

5.7 Problema de Aprisionamento

Durante os testes iniciais foi identificado um problema importante: o robô frequentemente ficava preso próximo aos obstáculos.

O comportamento observado incluía:

- repetição contínua de movimentos;
- excesso de ré;
- rotação sem avanço;
- colisões repetidas;
- dificuldade para sair de regiões fechadas.

Esse foi um dos principais desafios do projeto.

5.8 Implementação do Sistema de Fuga

Para resolver o problema de aprisionamento foi criado um sistema de fuga dividido em etapas.

O sistema foi baseado em estados temporários de comportamento.

Etapa 1 — Ré

Ao detectar obstáculo próximo, o robô inicialmente realiza movimento de ré para se afastar da colisão.

Essa etapa permite criar espaço para realizar manobras posteriores.

Etapa 2 — Escolha da Direção

O algoritmo compara os sensores laterais para identificar qual lado possui maior espaço livre.

Com isso, o sistema define:

- direção de fuga para esquerda;
- ou direção de fuga para direita.

Etapa 3 — Rotação Forte

Após definir a direção livre, o robô realiza uma rotação mais intensa.

Essa etapa foi extremamente importante para evitar que o robô continuasse preso repetindo pequenas curvas.

Foram realizados diversos ajustes até encontrar uma rotação suficientemente forte para permitir a saída do obstáculo.

Etapa 4 — Contorno

Após a rotação, o robô avança realizando curvas até encontrar uma região livre.

Essa etapa ajuda o sistema a contornar obstáculos maiores.

Etapa 5 — Retorno ao Objetivo

Quando os sensores deixam de detectar obstáculos, o sistema retorna ao modo normal de navegação e volta a perseguir o Target.

6. Problemas Encontrados

Durante o desenvolvimento diversos problemas reais foram encontrados.

6.1 Robô Andando na Direção Errada

Inicialmente o robô se deslocava para trás em relação ao Target.

Solução

Foi necessário ajustar:

- cálculo do erro angular;
- sinais das rodas;
- conversão das velocidades.

6.2 Oscilação Excessiva

O robô realizava muitas rotações sem conseguir avançar corretamente.

Solução

Foram ajustados:

- limites de velocidade angular;
- velocidade linear;
- ganho de rotação.

6.3 Sensores Detectando Tarde

Em alguns testes os obstáculos eram detectados muito próximos.

Solução

Foi ajustada a margem de segurança dos sensores.

6.4 Robô Preso em Obstáculos

O principal problema do projeto foi o aprisionamento próximo aos obstáculos.

Solução

Foi desenvolvido o sistema completo de fuga baseado em:

- ré;
- escolha de direção;
- rotação forte;
- contorno;

- retorno ao objetivo.

Esse sistema melhorou significativamente o comportamento do robô.

7. Resultados Obtidos

Ao final do desenvolvimento o sistema foi capaz de:

- navegar autonomamente;
- localizar o Target;
- evitar obstáculos;
- realizar curvas;
- controlar diferencialmente as rodas;
- escapar de regiões presas;
- encerrar automaticamente a simulação ao alcançar o objetivo.

Os testes demonstraram melhora progressiva do comportamento do robô após os ajustes realizados durante o desenvolvimento.

O projeto atingiu os objetivos propostos da disciplina.

8. Conclusão

O projeto permitiu aplicar conceitos importantes de:

- robótica móvel;
- navegação autônoma;
- sensores;
- controle diferencial;
- tomada de decisão;
- desvio de obstáculos;
- integração entre simulador e linguagem de programação.

Além da implementação do algoritmo, o projeto também exigiu diversos ajustes práticos relacionados ao comportamento real do robô em situações complexas de navegação.

A criação do sistema de fuga foi um dos pontos mais importantes do desenvolvimento, permitindo resolver problemas de aprisionamento e melhorar significativamente a autonomia do sistema.

O projeto proporcionou grande aprendizado sobre navegação robótica e integração entre Python e CoppeliaSim.

9. Referências

FOX, Dieter; BURGARD, Wolfram; THRUN, Sebastian.
The Dynamic Window Approach to Collision Avoidance.

PythonRobotics — Dynamic Window Approach.

Documentação oficial do CoppeliaSim.

Documentação oficial da ZMQ Remote API.

Material da disciplina de Robótica I.